Projet DreamPark : Partie 0

Spécification fonctionnelle et spécification des tests

Équipe : 404 Places Not Found
Membres : Japhet KOYAKOSSO ESSO (spécification des tests), Ayachi Mohamed (spécification fonctionnelle)
Formation : L3 MIASHS, parcours Informatique et SHS
Université Toulouse 2 Jean-Jaurès

---

1. Le projet en quelques mots

DreamPark est un parking automatisé. Il a deux accès, et chacun est équipé d'une caméra, d'une borne à tickets-paiement, d'un panneau d'affichage et de deux téléporteurs (un pour l'entrée, un pour la sortie). Chaque place a un identifiant unique formé de son niveau et de son numéro (« A1 », « B12 »…), ainsi qu'une longueur et une hauteur.

Quand une voiture arrive, la caméra relève son immatriculation et ses dimensions, puis le système lui attribue une place assez grande. Le téléporteur la conduit à cette place. Quand elle repart, la place est libérée et les panneaux sont mis à jour.

En plus du stationnement simple, le parking propose des abonnements. Un abonné peut demander une maintenance, un entretien ou la livraison de sa voiture à une adresse et une heure données. Il existe aussi un pack garantie : le parking s'engage à trouver une place au client, quitte à garer sa voiture dans un autre parking sans qu'il s'en rende compte. Enfin, le système garde une trace de chaque passage pour produire des statistiques.

Cette Partie 0 décrit ce que le système doit faire et comment nous allons le tester. Il n'y a pas encore d'implémentation.

---

2. Les acteurs

Les acteurs se rangent en quatre catégories : acteurs principaux, acteurs secondaires, matériel externe et autres systèmes.

Acteur	Catégorie	Ce qu'il fait
Client	Principal	Se gare, reprend sa voiture, peut s'abonner ou s'inscrire au pack garantie
Abonné	Principal	Comme le client, et peut en plus demander des services ou se désabonner
Super_Abonné	Principal	Client inscrit au pack

Acteur	Catégorie	Ce qu'il fait
		garantie, peut s'en désinscrire
Voiturier	Secondaire	Effectue les livraisons
Administrateur	Secondaire	Consulte et édite les statistiques

Abonné et Super_Abonné sont des spécialisations de Client : un sous-acteur peut faire tout ce que fait l'acteur parent, et d'autres choses en plus.

La caméra, la borne, le panneau et les téléporteurs ne sont pas représentés comme acteurs dans le diagramme fourni. Ils font partie du système et apparaissent dans le diagramme de classes.

Une remarque sur le modèle fourni : d'après l'énoncé, le pack garantie donne accès aux services des abonnés. Pourtant, sur le diagramme, Super_Abonné hérite de Client et non d'Abonné, donc il n'est relié à aucun cas du paquetage SERVICES. Nous gardons le diagramme tel quel. Dans l'implémentation, un super-abonné aura quand même accès aux services, puisque la note du diagramme précise que « pack garantie et abonnement désignent la même chose ».

---

3. Les cas d'utilisation

Paquetage	Cas d'utilisation	Acteur
CLIENT	Se garer	Client
CLIENT	Reprendre la voiture	Client
ABONNEMENT	S'abonner	Client
ABONNEMENT	Se désabonner	Abonné
PACK_GARANTIE	S'inscrire au pack garantie	Client
PACK_GARANTIE	Se désinscrire du pack garantie	Super_Abonné
SERVICES	Demander_Maintenance	Abonné
SERVICES	Demander_Entretien	Abonné
SERVICES	Demander_Livraison	Abonné
SERVICES	Effectuer la livraison	Voiturier
ADMINISTRATION	Consulter les statistiques	Administrateur
ADMINISTRATION	Éditer les statistiques	Administrateur

C'est bien le Client qui s'inscrit au pack garantie : on ne devient Super_Abonné qu'une fois inscrit.

Relations entre cas d'utilisation

Une extension signifie que B est une partie optionnelle de A (« B étend A »). Le diagramme fourni en contient quatre :

- S'abonner <<extend>> Se garer : quand il se gare, un client non abonné se voit proposer les abonnements et peut en choisir un.
- S'inscrire au pack garantie <<extend>> Se garer : de la même façon, il peut choisir la formule pack garantie.
- Effectuer la livraison <<extend>> Demander_Livraison : la livraison n'a lieu que si elle a été demandée.
- Éditer les statistiques <<extend>> Consulter les statistiques : l'administrateur peut éditer ce qu'il consulte, sans y être obligé.

Le diagramme ne contient aucune relation <<include>>.

---

4. Description des cas d'utilisation

4.1 Se garer

Acteur : Client.

But : garer sa voiture sur une place qui lui convient.

Le cas commence quand le client arrive devant un des deux accès. Il se termine quand la voiture est téléportée, ou quand le client apprend qu'il n'y a pas de place pour lui.

Préconditions : la voiture est devant un accès et n'est pas déjà dans le parking.

Scénario nominal (client non abonné) : 1. Le client arrive devant l'accès. 2. La caméra relève l'immatriculation, la hauteur et la longueur de la voiture. 3. La borne demande le statut du client, puis lit et analyse sa carte s'il en a une. 4. Le client n'est pas super-abonné : le parking cherche une place libre assez grande. 5. Une place est trouvée. La borne vérifie si le client est abonné. 6. Il ne l'est pas : la borne lui demande son mode de paiement (CB ou espèces). 7. La borne lui propose les abonnements, puis récupère son choix (il peut n'en prendre aucun). 8. La borne délivre un ticket. 9. Le téléporteur conduit la voiture à la place attribuée. 10. La place passe à « occupée » et les panneaux des deux accès affichent le nouveau nombre de places libres.

Variante, le client est abonné : aux étapes 6 et 7, la borne lui propose la liste des services (maintenance, entretien, livraison) et récupère son choix. La suite ne change pas.

Variante, le client est super-abonné : après l'analyse de la carte, il n'y a ni recherche de place ni ticket. Le téléporteur prend directement la voiture en charge. Si le parking est plein, elle est garée dans un parking partenaire sans que le client le voie.

Variante, aucune place ne convient : la borne affiche « Aucune place disponible pour votre véhicule ». Aucun ticket n'est délivré et la voiture n'entre pas.

Postconditions en cas de succès : un Placement en cours relie la voiture à sa place, la place n'est plus libre, la voiture est dans le parking et les panneaux sont à jour.

4.2 Reprendre la voiture

Acteur : Client.

Le cas commence quand le client se présente à un accès avec son ticket. Il se termine quand la voiture lui est rendue.

Préconditions : la voiture est dans le parking et le client a le ticket correspondant.

Scénario nominal : 1. Le client présente son ticket à la borne. 2. Le système retrouve la voiture et sa place. 3. Le téléporteur ramène la voiture à l'accès. 4. Le placement est clôturé (date de fin renseignée) et la place redevient libre. 5. Le téléporteur informe le parking que la place est à nouveau disponible. 6. Les panneaux affichent le nouveau nombre de places libres.

Postconditions : le placement n'est plus en cours mais il est conservé, puisqu'il sert de trace de passage pour les statistiques. La voiture n'est plus dans le parking.

Variante, livraison : si le client a demandé une livraison, c'est le système qui reprend la voiture, et un voiturier la livre à l'adresse et à l'heure prévues.

Variante, maintenance ou entretien : le système sort la voiture, fait réaliser le service, puis la regare. Un nouveau placement est créé, sans intervention du client.

4.3 S'abonner

Acteur : Client. 1. Le client consulte les formules proposées (à la borne, pendant « Se garer »). 2. Il en choisit une. 3. Le système crée un contrat entre le client et l'abonnement, avec la date du jour comme date de début. 4. Le client est désormais abonné et a accès aux services.

Si la formule choisie est un pack garantie, le client devient aussi super-abonné. Un client n'a qu'un abonnement à la fois (multiplicité 0..1 de `monAbonnement`) : s'il en prend un nouveau, l'ancien contrat est rompu.

4.4 Se désabonner

Acteur : Abonné. 1. L'abonné demande la résiliation. 2. Le système retrouve son contrat en cours et le rompt (la date de fin est renseignée). 3. Le client n'est plus abonné. Comme ce statut est calculé à partir des contrats, il n'y a rien d'autre à modifier.

4.5 Pack garantie

S'inscrire (acteur : Client) : c'est un abonnement de type pack garantie (`estPackGar` vrai). Le client devient super-abonné. Le parking lui garantit une place, et si DreamPark est plein, sa voiture est garée ailleurs sans qu'il ait à s'en occuper. Il a aussi accès aux services des abonnés.

Se désinscrire (acteur : Super_Abonné) : même déroulement que « Se désabonner ». Le contrat pack garantie est rompu et le client n'est plus super-abonné.

4.6 Demander une maintenance ou un entretien

Acteur : Abonné. 1. L'abonné fait sa demande. Elle est enregistrée avec sa date. 2. Le système sort la voiture de sa place. 3. La maintenance ou l'entretien est réalisé, avec sa date et un rapport. 4. La voiture est regarée (nouveau placement).

Un client sans abonnement en cours ne peut pas faire de demande.

4.7 Demander une livraison

Acteur : Abonné. Informations nécessaires : date, heure et adresse. 1. L'abonné demande la livraison de sa voiture. 2. La demande est enregistrée et affectée à un voiturier. 3. Le jour venu, la voiture est sortie du parking et le voiturier la livre.

Politique de flexibilité : tant que la livraison n'est pas faite, le client peut changer l'adresse ou l'heure par simple appel. Concrètement, une nouvelle demande remplace celle qui est en attente.

4.8 Effectuer la livraison

Acteur : Voiturier. Il prend la voiture et la livre à l'adresse et à l'heure de la demande. Il ne peut pas livrer une voiture pour laquelle il n'existe aucune demande.

4.9 Consulter et éditer les statistiques

Acteur : Administrateur. Le système garde une trace de chaque passage, c'est-à-dire de chaque placement (voiture, place, date de début, date de fin), ainsi que des services demandés. L'administrateur peut consulter la fréquentation et l'activité par service, puis éditer ces informations sous différentes formes (texte, HTML, image pour les plans, vidéo). D'après l'organisation du projet, cette partie sera réalisée en Partie 3 ; l'interface graphique viendra en Partie 4.

---

5. Diagramme de classes

Nous reprenons le diagramme fourni avec sa notation : + public, # protégé, - privé, / pour un attribut dérivé (calculé, jamais affecté directement), et un attribut souligné pour un attribut de classe.

Classe	Attributs	Opérations
Parking (singleton)	`- nbPlacesParNiveau : int`, `- /nbPlacesLibres : int`, `- prix : int` (attribut de classe), `+ nbNiveaux : int`	`+ rechercherPlace(v : Voiture) : Place`, `+ nbPlacesLibresParNiveau(niveau : char) : int`, `+ addAbonnement(ab : Abonnement)`, `_reinitialiser()` (ajout réservé aux tests, voir §7)
Place	`+ numero : int`, `-`	`+ addPlacementP(p :`

Classe	Attributs	Opérations
	niveau : char, - longueur : float, - hauteur : float, - estLibre : bool	Placement)
Voiture	- immatriculation : string, - hauteur : float, - longueur : float, + estDansParking : bool, + lieuStationnement : string (ajout, voir §7)	+ addPlacementV(p : Placement)
Placement	+ dateDebut : Date, + dateFin : Date, + /estEnCours : bool	+ partirPlace()
Client	+ nom : string, + adresse : string, + /estAbonne : bool, + /estSuperAbonne : bool, + /nbFrequentations : int	+ sAbonner(ab : Abonnement), + nouvelleVoiture(imma : string, hautV : float, longV : float), + seDesabonner(), + demanderMaintenance(), + demanderEntretien(), + demanderLivraison(dateLiv : Date, heure : int, adresseLiv : string), + entrerParking(a : Acces) : string
Abonnement	+ libelle : string, + prix : float, + estPackGar : bool	+ addContrat(contrat : Contrat)
Contrat	+ dateDebut : Date, + dateFin : Date, + /estEnCours : bool	+ rompreContrat()
Service	+ dateDemande : Date, + dateService : Date, + rapport : string	
Maintenance (hérite de Service)		+ effectuerMaintenance(v : Voiture)
Entretien (hérite de Service)		+ effectuerEntretien()
Livraison (hérite de Service)		+

Classe	Attributs	Opérations
		effectuerLivraison()
Voiturier	- numVoiturier : int	+ livrerVoiture(v : Voiture, date : Date, heure : int)
Acces		+ actionnerCamera(c : Client) : Voiture,+ actionnerPanneau() : string,+ lancerProcedureEntree(c : Client) : string
Camera		+ capturerHauteur(v : Voiture) : float,+ capturerLongueur(v : Voiture) : float,+ capturerImmatr(v : Voiture) : string
BorneTicket		+ delivrerTicket(c : Client) : string,+ proposerServices() : string,+ proposerAbonnements(c : Client, p : Parking) : string,+ recupererInfosCarte(c : Client) : string,+ proposerTypePaiement() : string
PanneauAffichage		+ afficherNbPlacesDisponibles(p : Parking) : string
Teleporteur		+ teleporterVoiture(v : Voiture, p : Place) : Placement,+ teleporterVoitureSuperAbonne(v : Voiture) : string

Comment les attributs dérivés sont calculés : - `Placement.estEnCours` et `Contrat.estEnCours` sont vrais tant que `dateFin` n'est pas renseignée. - `Client.estAbonne` est vrai si le client a un contrat en cours. - `Client.estSuperAbonne` est vrai si ce contrat porte sur un abonnement dont `estPackGar` est vrai. - `Client.nbFrequentations` est le nombre de placements de la voiture du client. - `Parking.nbPlacesLibres` est le nombre de places dont `estLibre` est vrai.

En Python, ces attributs seront des `property`. Les attributs privés suivront la convention `__attr` pour privé et `_attr` pour protégé.

Associations

- Placement est une classe-association entre Voiture (0..1) et Place (*) : une place voit passer plusieurs voitures dans le temps, et chaque passage est un placement.
- Contrat est une classe-association entre Client (*) et Abonnement (0..1, rôle `monAbonnement`).
- Parking est composé de ses places, avec une association avec restriction par la clé `idPlace : string`. C'est cette clé (niveau + numéro, par exemple « A1 ») qui rend l'identifiant unique.
- Parking est composé des abonnements qu'il propose (`mesAbonnements`).
- Client est composé de ses services (`mesServices`).
- Client 1 — 1 Voiture (rôles `proprietaire` et `maVoiture`).
- Client * — 1 Acces : le client passe par un accès.
- Parking 1 — 2 Acces (`acces1`, `acces2`).
- Acces agrège une Camera, une BorneTicket, un PanneauAffichage et deux Teleporteur (`TelEntree`, `TelSortie`).
- Livraison 0..* — 1 Voiturier.

Comme l'indique la note du diagramme, le voiturier n'intervient pas dans « Se garer ».

---

6. Règles fonctionnelles

#	Règle
RF01	La caméra relève l'immatriculation, la hauteur et la longueur de la voiture.
RF02	Le système attribue une place libre dont la hauteur et la longueur sont supérieures ou égales à celles de la voiture.
RF03	Si aucune place ne convient, la voiture n'est pas garée et le client en est informé.
RF04	Chaque entrée et chaque sortie mettent à jour le nombre de places libres et les deux panneaux.
RF05	Un ticket est délivré seulement si une place a été attribuée.
RF06	Seuls les clients ayant un abonnement en cours ont accès aux services (maintenance, entretien, livraison).
RF07	Un super-abonné a toujours une solution de stationnement, éventuellement dans un autre parking.

#	Règle
RF08	Une livraison est demandée avec une date, une heure et une adresse, puis effectuée par un voiturier.
RF09	Chaque passage est conservé pour étudier la fréquentation.
RF10	L'administrateur peut consulter et éditer les statistiques.
RF11	Le client non abonné choisit son mode de paiement : CB ou espèces.
RF12	Un client peut s'abonner et se désabonner ; il a au plus un abonnement en cours.
RF13	Le client peut modifier sa demande de service tant qu'elle n'a pas été exécutée.
RF14	Une place est identifiée de façon unique par son niveau et son numéro.

7. Choix de conception

Nous avons essayé de rester au plus près du modèle fourni. Chaque écart est justifié ci-dessous, comme le demande la consigne.

Parking plein. `rechercherPlace` renvoie `None`. L'énoncé dit qu'une « valeur particulière est retournée », et un parking plein est une situation normale, pas une erreur.

Une seule classe Client. Il n'y a pas de sous-classes Abonné et Super_Abonné, car le diagramme de classes ne contient qu'une classe CLIENT. La hiérarchie n'existe que du côté des acteurs. Le statut est calculé par `estAbonne` et `estSuperAbonne`.

Gestion des erreurs. Nous appliquons une règle unique partout : - un résultat normal du métier (parking plein, client non abonné, contrat déjà rompu, place déjà libre) donne une valeur de retour : `None` ou `False` ; - un appel incohérent, qui ne devrait jamais arriver si le reste du programme est correct, lève une exception. Nous définirons `PlaceOccupeeError`, `PlaceIncompatibleError`, `VoitureDejaGareeError` et `LivraisonSansDemandeError`, et nous utiliserons `ValueError` pour une donnée invalide (dimension négative, heure hors de 0 à 23, niveau inexistant).

Ces exceptions sont un ajout au modèle. Elles ne changent ni les classes métier ni leurs associations.

Dimensions égales. Une voiture de 2,00 m entre sur une place de 2,00 m : la comparaison se fait avec « inférieur ou égal ».

Statistiques. Nous n'ajoutons pas de classe Statistiques. Les placements clôturés sont conservés et forment déjà la trace de passage ; `nbFrequentations` en est dérivé. Il faudra quand même une opération pour récupérer cet historique depuis Parking (par exemple

`historiquePassages()`). Comme elle n'existe pas dans le diagramme, c'est un ajout que nous justifions par RF09 et RF10. L'édition sous différents formats relève de la vue du patron MVC et sera traitée en Partie 3.

Retour de la voiture. Le diagramme ne donne pas au téléporteur d'opération pour ramener une voiture. C'est `Placement.partirPlace()` qui clôture le placement et libère la place ; le téléporteur s'en sert lors de la sortie.

Ticket. Nous n'ajoutons pas de classe Ticket : dans le diagramme fourni, `delivrerTicket` renvoie une chaîne, et nous gardons ce choix. Le ticket est une chaîne unique construite à partir de l'identifiant de la place, de l'immatriculation et de l'heure d'entrée (par exemple « A1-AB123CD-0930 »). Elle suffit pour retrouver le placement en cours lors de « Reprendre la voiture ». Si un ticket devait porter plus d'informations (prix, mode de paiement), nous ajouterions une classe Ticket et nous justifierions cet écart.

Parking partenaire. Le diagramme ne représente pas le parking où l'on gare un super-abonné quand DreamPark est plein ; `teleporterVoitureSuperAbonne` renvoie seulement une chaîne. Pour pouvoir vérifier RF07, nous ajoutons à Voiture un attribut `lieuStationnement : string` qui vaut l'identifiant de la place (« A1 ») ou « PARTENAIRE ». C'est un écart au modèle, justifié par le besoin de savoir où se trouve la voiture et de la rendre au client.

Singleton et tests. Parking étant un singleton, la même instance serait partagée par tous les tests, qui s'influenceraient. Nous ajoutons donc une méthode de classe réservée aux tests, `Parking._reinitialiser()`, qui supprime l'instance unique. Elle est appelée dans le `tearDown()` de chaque classe de test, si bien que chaque `setUp()` repart d'un parking neuf. C'est un ajout au modèle, limité aux tests.

Persistance. Le format de sauvegarde sera choisi lors de l'implémentation (point 3.2 du travail à réaliser). Il ne bloque pas la spécification.

---

8. Spécification des tests unitaires

Nous prévoyons une classe de test `unittest.TestCase` par classe métier, un `setUp()` qui prépare le contexte, et une méthode `test_...` par cas. Pour chaque cas, nous indiquons le résultat attendu et l'assertion qui le vérifiera. Les classes de test seront documentées avec des docstrings (`:param`, `:return`) pour générer la documentation avec pydoc, comme dans `docGen.py`.

Démarche

Ce sont des tests fonctionnels (boîte noire) : ils sont écrits à partir de la spécification, avant le code. Pour choisir les données de test, nous procédons en trois temps : - Classes d'équivalence. Nous découpons les entrées en groupes qui doivent se comporter pareil, sans oublier les cas invalides. Pour la compatibilité voiture-place : voiture plus petite que la place, de mêmes dimensions, plus grande, dimension négative ou nulle. Pour le statut du client : non abonné, abonné, super-abonné. - Valeurs limites. Pour chaque borne, nous testons la valeur elle-même et ses voisines immédiates. Exemples : une place de 2,00 m avec des voitures de 1,99 m, 2,00 m

et 2,01 m ; un parking avec 0, 1 et toutes ses places libres ; une heure de livraison à -1, 0, 23 et 24. - Oracle. Chaque cas a son résultat attendu, fixé d'après la spécification et non d'après le code, avec l'assertion qui le vérifie.

Niveaux de test. Les classes 8.1 à 8.12 sont des tests unitaires : chaque classe est testée seule. Quand elle dépend d'un composant matériel ou absent (la caméra, le parking partenaire), nous le remplaçons par un bouchon (stub) qui renvoie des valeurs fixées. TestAcces (8.13) est un test d'intégration : il vérifie que caméra, borne, parking, téléporteur et panneau coopèrent sur le scénario « Se garer ». Enfin, tous les tests seront relancés après chaque modification (non-régression), par exemple avant chaque commit sur le dépôt git.

Chaque classe de test appelle `Parking._reinitialiser()` dans son `tearDown()` pour que les tests restent indépendants (voir les choix de conception).

Contexte commun, sauf mention contraire : un parking avec un niveau A de trois places. A1 et A2 mesurent 2,00 m de haut sur 5,00 m de long ; A3 mesure 1,60 m sur 4,00 m. Une voiture V mesure 1,80 m sur 4,50 m (immatriculation « AB-123-CD »). Le client C n'est pas abonné. Deux abonnements sont disponibles : « Mensuel » (pas pack garantie) et « Garantie » (pack garantie).

8.1 TestParking

Méthode testée	Cas	Résultat attendu	Assertion	RF
`Parking()`	Deux instanciations	Même objet	`assertIs(p1, p2)`	
`Parking._reinitialiser()`	Après réinitialisation	`Parking()` renvoie un parking neuf, sans voiture ni abonnement	`assertEqual(p.nbPlacesLibres, ...)`	
`rechercherPlace(v)`	V, A1 libre	Renvoie A1 ou A2	`assertIn(place, [A1, A2])`	RF02
`rechercherPlace(v)`	V trop haute pour toutes les places (2,10 m)	`None`	`assertIsNone`	RF03
`rechercherPlace(v)`	A1 et A2 occupées, V ne rentre pas dans A3	`None`	`assertIsNone`	RF03
`rechercherPlace(v)`	Voiture de 2,00 m × 5,00 m (dimensions égales)	Renvoie A1 ou A2	`assertIsNotNone`	RF02
`rechercherPlace(v)`	Voiture de 1,99 m × 4,99 m (juste en	Renvoie A1 ou A2	`assertIsNotNone`	RF02

Méthode testée	Cas	Résultat attendu	Assertion	RF
	dessous)			
`rechercherPlace(v)`	Voiture de 2,01 m × 4,50 m (juste au-dessus en hauteur)	`None`	`assertIsNone`	RF03
`rechercherPlace(v)`	Voiture de 1,80 m × 5,01 m (juste au-dessus en longueur)	`None`	`assertIsNone`	RF03
`rechercherPlace(v)`	Une seule place libre et compatible	Renvoie cette place	`assertIs`	RF02
`nbPlacesLibresParNiveau('A')`	Aucune voiture	3	`assertEqual(..., 3)`	RF04
`nbPlacesLibresParNiveau('A')`	Après une entrée	2	`assertEqual(..., 2)`	RF04
`nbPlacesLibresParNiveau('Z')`	Niveau inexistant	Exception	`assertRaises(ValueError)`	
`nbPlacesLibres`	Après une entrée et une sortie	3	`assertEqual`	RF04
`addAbonnement(ab)`	Nouvel abonnement	Présent dans `mesAbonnements`	`assertIn`	RF12
`addAbonnement(ab)`	Abonnement déjà présent	Pas ajouté deux fois	`assertEqual(len(...), 2)`	
Clé `idPlace`	Place A1	`"A1"`	`assertEqual`	RF14
Clé `idPlace`	Ajout d'une deuxième place A1	Refusé	`assertRaises(ValueError)`	RF14

8.2 TestPlace

Méthode testée	Cas	Résultat attendu	Assertion	RF
`addPlacementP(p)`	Place libre, voiture compatible	La place n'est plus libre	`assertFalse(place.estLibre)`	RF02
`addPlacementP(p)`	Place déjà occupée	Exception	`assertRaises(PlaceOccupeeError)`	RF02

Méthode testée	Cas	Résultat attendu	Assertion	RF
`addPlacementP(p)`	Voiture plus grande que la place	Exception	`assertRaises(PlaceIncompatibleError)`	RF02
Constructeur	Hauteur négative	Exception	`assertRaises(ValueError)`	

8.3 TestVoiture

Méthode testée	Cas	Résultat attendu	Assertion	RF
`addPlacementV(p)`	Voiture hors du parking	`estDansParking` vrai	`assertTrue`	RF04
`addPlacementV(p)`	Voiture déjà garée	Exception	`assertRaises(VoitureDejaGareeError)`	RF03
Constructeur	Immatriculation vide	Exception	`assertRaises(ValueError)`	RF01

8.4 TestPlacement

Méthode testée	Cas	Résultat attendu	Assertion	RF
Création	Nouveau placement	`estEnCours` vrai, `dateFin` vide	`assertTrue`, `assertIsNone`	
`partirPlace()`	Placement en cours	`dateFin` renseignée, place libre, voiture hors du parking	`assertFalse(p.estEnCours)`, `assertTrue(place.estLibre)`	RF04
`partirPlace()`	Placement déjà terminé	Aucun effet, renvoie `False`	`assertFalse`	RF04
`partirPlace()`	Après la sortie	Le placement est conservé dans l'historique	`assertIn(p, parking.historiquePassages())`	RF09

8.5 TestClient

Méthode testée	Cas	Résultat attendu	Assertion	RF
`nouvelleVoiture(imma, h, l)`	Données valides	`maVoiture` a la bonne immatriculation	`assertEqual`	
`nouvelleVoiture(imma, h, l)`	Longueur négative	Exception	`assertRaises(ValueError)`	
`sAbonner(ab)`	Formule «	`estAbonne`	`assertTrue`,	RF12

Méthode testée	Cas	Résultat attendu	Assertion	RF
	Mensuel »	vrai, `estSuperAbonne` faux	`assertFalse`	
`sAbonner(ab)`	Formule « Garantie »	`estAbonne` et `estSuperAbonne` vrais	`assertTrue` ×2	RF07
`sAbonner(ab)`	Déjà abonné, nouvelle formule	Ancien contrat rompu, un seul contrat en cours	`assertEqual(nbContratsEnCours, 1)`	RF12
`seDesabonner()`	Client abonné	`estAbonne` faux	`assertFalse`	RF12
`seDesabonner()`	Client non abonné	Renvoie `False`	`assertFalse`	RF12
`demanderMaintenance()`	Client abonné	Un service Maintenance ajouté à `mesServices`	`assertIsInstance(..., Maintenance)`	RF06
`demanderMaintenance()`	Client non abonné	Renvoie `False`, aucun service ajouté	`assertFalse`, `assertEqual(len(mesServices), 0)`	RF06
`demanderEntretien()`	Client abonné / non abonné	Mêmes résultats que la maintenance	idem	RF06
`demanderLivraison(d, h, adr)`	Client abonné	Livraison enregistrée avec date, heure et adresse	`assertEqual` ×3	RF08
`demanderLivraison(d, h, adr)`	Deuxième demande avant exécution	La nouvelle adresse remplace l'ancienne	`assertEqual(livraison.adresse, nouvelleAdr)`	RF13
`demanderLivraison(d, h, adr)`	Heure 0 puis 23 (bornes valides)	Livraison enregistrée	`assertEqual`	RF08
`demanderLivraison(d, h, adr)`	Heure -1 puis 24 (juste hors bornes)	Exception	`assertRaises(ValueError)`	RF08
`demanderLivraison(d, h, "")`	Adresse vide	Exception	`assertRaises(ValueError)`	RF08
`entrerParking(a)`	Place disponible	Renvoie un numéro de	`assertIsNotNone`	RF05

Méthode testée	Cas	Résultat attendu	Assertion	RF
		ticket		
`entrerParking(a)`	Parking plein	Renvoie le message d'absence de place	`assertEqual(..., "Aucune place disponible pour votre véhicule")`	RF03
`nbFrequentations`	Deux entrées et deux sorties	2	`assertEqual`	RF09

8.6 TestAbonnement et TestContrat

Méthode testée	Cas	Résultat attendu	Assertion	RF
`Abonnement.addContrat(c)`	Nouveau contrat	Contrat rattaché à l'abonnement	`assertIn`	RF12
Création d'un contrat		`dateDebut` = aujourd'hui, `estEnCours` vrai	`assertEqual`, `assertTrue`	RF12
`Contrat.rompreContrat()`	Contrat en cours	`dateFin` renseignée, `estEnCours` faux	`assertFalse`	RF12
`Contrat.rompreContrat()`	Contrat déjà rompu	Aucun effet, renvoie `False`	`assertFalse`	

8.7 TestMaintenance, TestEntretien et TestLivraison

Méthode testée	Cas	Résultat attendu	Assertion	RF
Création d'un service		`dateDemande` = aujourd'hui	`assertEqual`	RF09
`Maintenance.effectuerMaintenance(v)`	Voiture garée	`dateService` et `rapport` renseignés, voiture regarée (nouveau placement)	`assertIsNotNone`, `assertTrue(v.estDansParking)`	RF06
`Entretien.effectuerEntretien()`	Voiture garée	Mêmes résultats	idem	RF06
`Livraison.effectuerLivraison()`	Demande enregistrée	Le voiturier est appelé, la place est libérée, la voiture est hors du parking	`assertFalse(v.estDansParking)`	RF08

8.8 TestVoiturier

Méthode testée	Cas	Résultat attendu	Assertion	RF
`livrerVoiture(v, date, heure)`	Une demande existe pour V	Livraison marquée effectuée à la bonne date et à la bonne heure	`assertEqual`	RF08
`livrerVoiture(v, date, heure)`	Aucune demande pour V	Exception	`assertRaises(LivraisonSansDemandeError)`	RF08

8.9 TestCamera

Pour les tests, la caméra est simulée : elle lit les valeurs d'un objet Voiture.

Méthode testée	Cas	Résultat attendu	Assertion	RF
`capturerHauteur(v)`	V	1.80	`assertEqual`	RF01
`capturerLongueur(v)`	V	4.50	`assertEqual`	RF01
`capturerImmatr(v)`	V	`"AB-123-CD"`	`assertEqual`	RF01
`capturerImmatr(v)`	Plaque illisible (vide)	Exception	`assertRaises(ValueError)`	RF01

8.10 TestBorneTicket

Méthode testée	Cas	Résultat attendu	Assertion	RF
`delivrerTicket(c)`	Place attribuée	Un ticket non vide	`assertTrue(len(ticket) > 0)`	RF05
`delivrerTicket(c)`	Aucune place attribuée	`None`	`assertIsNone`	RF05
`delivrerTicket(c)`	Deux clients	Deux tickets différents	`assertNotEqual`	RF05
`delivrerTicket(c)`	Place A1, voiture V	Le ticket contient l'identifiant de la place et l'immatriculation	`assertIn("A1", ticket)`	RF05
`recupererInfosCarte(c)`	Client super-abonné	Indique le statut super-abonné	`assertIn("super", ...)`	RF07
`recupererInfosCarte(c)`	Client sans carte	Chaîne vide	`assertEqual(..., "")`	

Méthode testée	Cas	Résultat attendu	Assertion	RF
`proposerServices()`		Mentionne maintenance, entretien et livraison	`assertIn` ×3	RF06
`proposerAbonnements(c, p)`	Parking avec deux formules	Les deux libellés apparaissent	`assertIn` ×2	RF12
`proposerTypePaiement()`		Mentionne CB et espèces	`assertIn` ×2	RF11

8.11 TestPanneauAffichage

Méthode testée	Cas	Résultat attendu	Assertion	RF
`afficherNbPlacesDisponibles(p)`	Parking vide	Le texte contient « 3 »	`assertIn("3", ...)`	RF04
`afficherNbPlacesDisponibles(p)`	Après une entrée par l'accès 1	Les panneaux des deux accès affichent « 2 »	`assertEqual(panneau1, panneau2)`	RF04
`afficherNbPlacesDisponibles(p)`	Parking plein	Le texte contient « 0 »	`assertIn("0", ...)`	RF03

8.12 TestTeleporteur

Méthode testée	Cas	Résultat attendu	Assertion	RF
`teleporterVoiture(v, place)`	Place attribuée	Renvoie un placement qui relie V à cette place	`assertIs(placement.place, place)`	RF02
`teleporterVoiture(v, None)`	Pas de place	Exception	`assertRaises(ValueError)`	RF03
`teleporterVoitureSuperAbonne(v)`	Place libre dans DreamPark	La voiture est garée : placement en cours, place occupée, `lieuStationnement` = identifiant de la place, une place libre de moins	`assertTrue(v.estDansParking)`, `assertFalse(place.estLibre)`, `assertEqual(v.lieuStationnement, place.idPlace)`	RF07
`teleporterVoitureSuperAb`	DreamPark plein	La voiture est prise en charge	`assertEqual(v.lieuStatio`	RF07

Méthode testée	Cas	Résultat attendu	Assertion	RF
`onne(v)`		par le parking partenaire : `lieuStationnement` = « PARTENAIRE », aucune place de DreamPark modifiée, compteur toujours à 0	`nnement, "PARTENAIRE"), assertEqual(parking.nbPlacesLibres, 0)`	
`teleporterVoitureSuperAbonne(v)`	DreamPark plein	Le message n'est pas celui du refus	`assertNotEqual(..., "Aucune place disponible pour votre véhicule")`	RF07

8.13 TestAcces (intégration)

Ces tests vérifient l'enchaînement complet de « Se garer » au niveau d'un accès. La caméra est remplacée par un bouchon qui renvoie les dimensions de V.

Méthode testée	Cas	Résultat attendu	Assertion	RF
`actionnerCamera(c)`	Client avec V	Renvoie une Voiture avec les bonnes valeurs	`assertEqual` ×3	RF01
`actionnerPanneau()`	Après une entrée	Même texte que `afficherNbPlacesDisponibles`	`assertEqual`	RF04
`lancerProcedureEntree(c)`	Client non abonné, place disponible	Ticket délivré, V garée, une place libre de moins	`assertIsNotNone, assertTrue(v.estDansParking)`	RF01, RF02, RF05
`lancerProcedureEntree(c)`	Parking plein	Message d'absence de place, pas de ticket	`assertEqual, assertFalse(v.estDansParking)`	RF03
`lancerProcedureEntree(c)`	Client super-abonné, place libre	Pas de ticket, voiture garée dans DreamPark	`assertIsNone(ticket), assertTrue(v.estDansParking)`	RF07

Méthode testée	Cas	Résultat attendu	Assertion	RF
`lancerProcedureEntree(c)`	Client super-abonné, DreamPark plein	Pas de refus, voiture au parking partenaire	`assertEqual(v.lieuStationnement, "PARTENAIRE")`	RF07

Couverture des règles

RF	Classes de test
RF01	TestCamera, TestVoiture, TestAcces
RF02	TestParking, TestPlace, TestTeleporteur, TestAcces
RF03	TestParking, TestVoiture, TestClient, TestPanneauAffichage, TestTeleporteur, TestAcces
RF04	TestParking, TestVoiture, TestPlacement, TestPanneauAffichage, TestAcces
RF05	TestClient, TestBorneTicket, TestAcces
RF06	TestClient, TestMaintenance, TestEntretien, TestBorneTicket
RF07	TestClient, TestBorneTicket, TestTeleporteur, TestAcces
RF08	TestClient, TestLivraison, TestVoiturier
RF09	TestPlacement, TestClient, services
RF10	Tests de la vue statistiques, spécifiés en Partie 3
RF11	TestBorneTicket
RF12	TestParking, TestClient, TestAbonnement, TestContrat
RF13	TestClient
RF14	TestParking

Annexe : exemple de classe de test documentée pour pydoc

Voici la forme que prendront nos classes de test. Seuls les docstrings sont écrits à ce stade ; le corps des tests viendra en Partie 1.

```
import unittest


class TestParking(unittest.TestCase):
    """
    Tests unitaires de la classe Parking.
```

```
    Contexte : un niveau A de trois places (A1 et A2 : 2,00 m x 5,00
m,
    A3 : 1,60 m x 4,00 m) et une voiture de 1,80 m x 4,50 m.
    """

    def setUp(self):
        """Crée le parking, ses trois places et la voiture de test."""

    def test_singleton(self):
        """Deux appels à Parking() renvoient le même objet
(assertIs)."""

    def test_rechercherPlace_parking_plein(self):
        """
        Quand aucune place ne convient, rechercherPlace renvoie None.

        :return: None attendu (assertIsNone)
        """


if __name__ == '__main__':
    unittest.main()
```